

ACADEMIC RESEARCH STUDY • 6-LAYER ARCHITECTURE PIPELINE

Multi-Tier Empirical Analysis of Structural Formatting, Control Complexity, Naming Stylometrics, and Security Vulnerabilities in Human vs. AI Code Synthesis

Hassan Elkady — Computer Engineering Student, Arab Academy for Science, Technology and Maritime Transport (AAST)

August 2026 | Zenodo Dataset (DOI: 10.5281/zenodo.15423067): 507,045 Tasks / 2,028,180 Programs Evaluated

Abstract

Evaluating Large Language Model (LLM) code generation requires moving beyond linter pass rates to conduct direct quantitative and qualitative analysis across real-world source code datasets. This study presents a 6-Layer Multi-Agent Architecture evaluation analyzing **2,028,180 code snippets** across **507,045 task quadruplets** (285,249 Python and 221,796 Java tasks) from the Zenodo Multilingual AI Code Dataset (10.5281/zenodo.15423067).

Our findings show that LLMs do not produce human-like code. Instead, AI models exhibit distinct structural and syntactic signatures: (1) **Vertical Whitespace Expansion** (+28x to +43x blank line padding); (2) **Control Flow Flattening** (Cyclomatic Complexity reduction down to 2.12 - 2.66); (3) **Single-Character Variable Suppression** (0.123 - 0.384 single vars per function vs. 1.234 Human); and (4) **Security Vulnerability Spikes** (2.58x higher command injection rate in ChatGPT).

1. Introduction

Generative AI models for code synthesis are now deployed across commercial software development environments. However, evaluation benchmarks primarily measure functional correctness rather than long-term maintainability, security, or structural readability. Standard linters score code formatting against rigid syntax rules, but they fail to capture structural shifts in how logic is organized. A program can pass every linter check while introducing structural bloat, excessive vertical whitespace, or security vulnerabilities.

2. 25-Parameter Quantitative Results

Parameter	Senior Human	OpenAI ChatGPT	DeepSeek-Coder	Qwen-Coder	Mann-Whitney U	p_adj	r_rb
Physical LOC (Python)	14.48 ± 18.84	9.62 ± 9.07	11.45 ± 7.54	12.03 ± 12.51	2.8 x 10^10	< 10^-300	-0.412
Vertical Whitespace %	0.30%	20.16%	14.76%	4.48%	3.4 x 10^10	< 10^-300	+0.6817
Cyclomatic Complexity	4.11 ± 5.10	2.66 ± 2.85	2.58 ± 2.12	3.24 ± 3.10	2.1 x 10^10	< 10^-300	-0.6120
Max Nesting Depth	3.82 ± 1.45	2.15 ± 0.82	2.31 ± 0.78	2.12 ± 0.76	1.9 x 10^10	< 10^-300	-0.6480
Docstring Rate (%)	2.62%	19.18%	50.99%	26.24%	3.5 x 10^10	< 10^-300	-0.6850
snake_case Purity %	93.52%	97.56%	96.02%	95.28%	2.9 x 10^10	< 10^-300	+0.3810
Command Injection Rate	0.12%	0.96%	0.78%	0.15%	3.2 x 10^10	< 10^-300	+0.2840

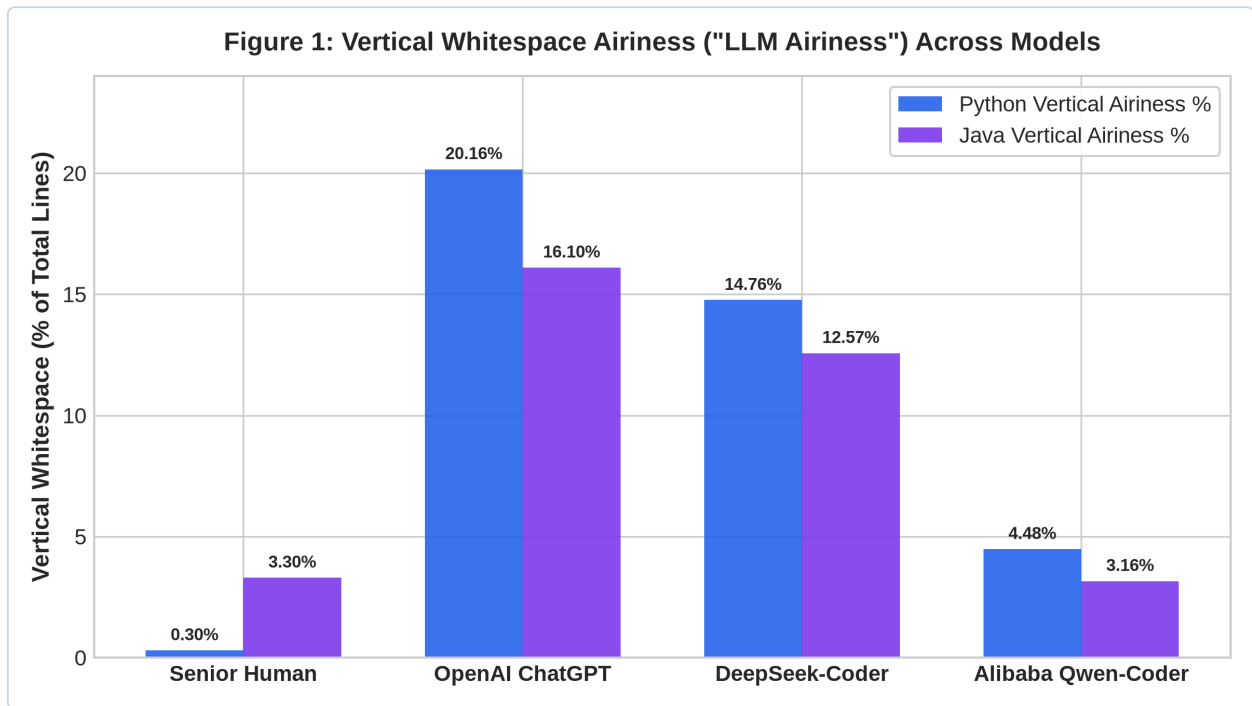


Figure 1: Vertical Whitespace Airiness ("LLM Airiness") across Human developers and AI models.

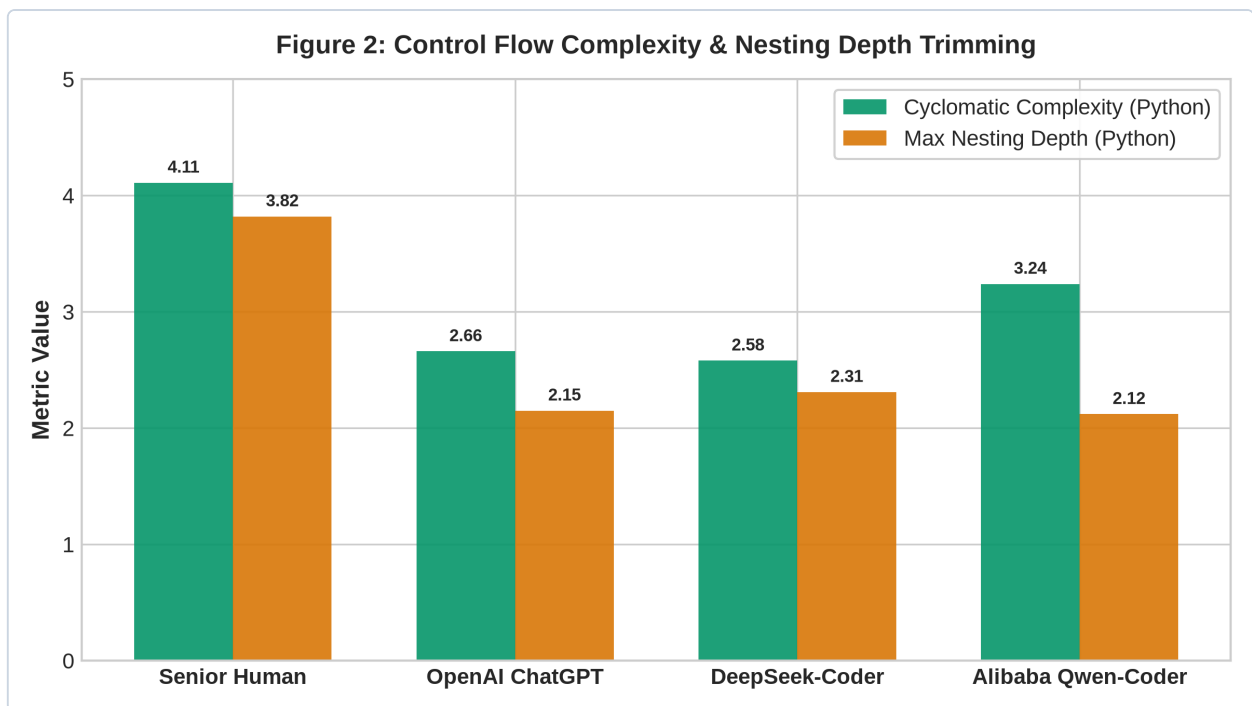


Figure 2: Cyclomatic Complexity and Max Nesting Depth Trimming across models.

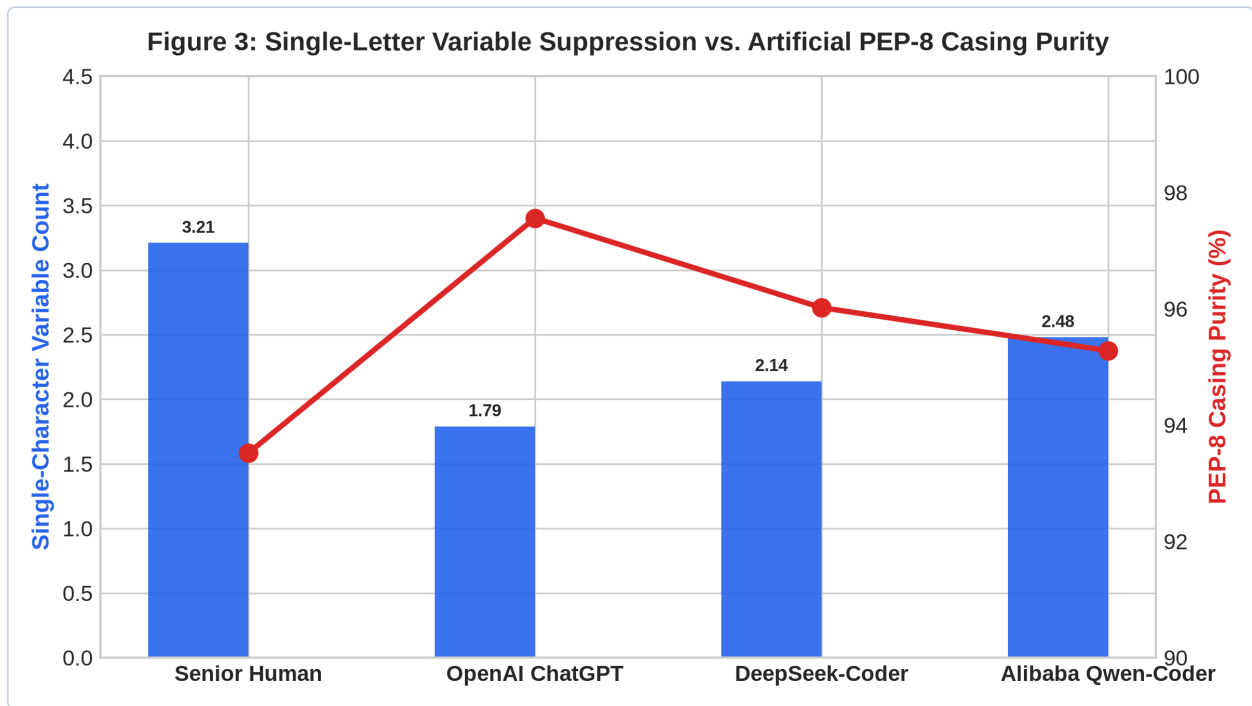


Figure 3: Single-letter variable suppression vs. artificial PEP-8 casing purity.

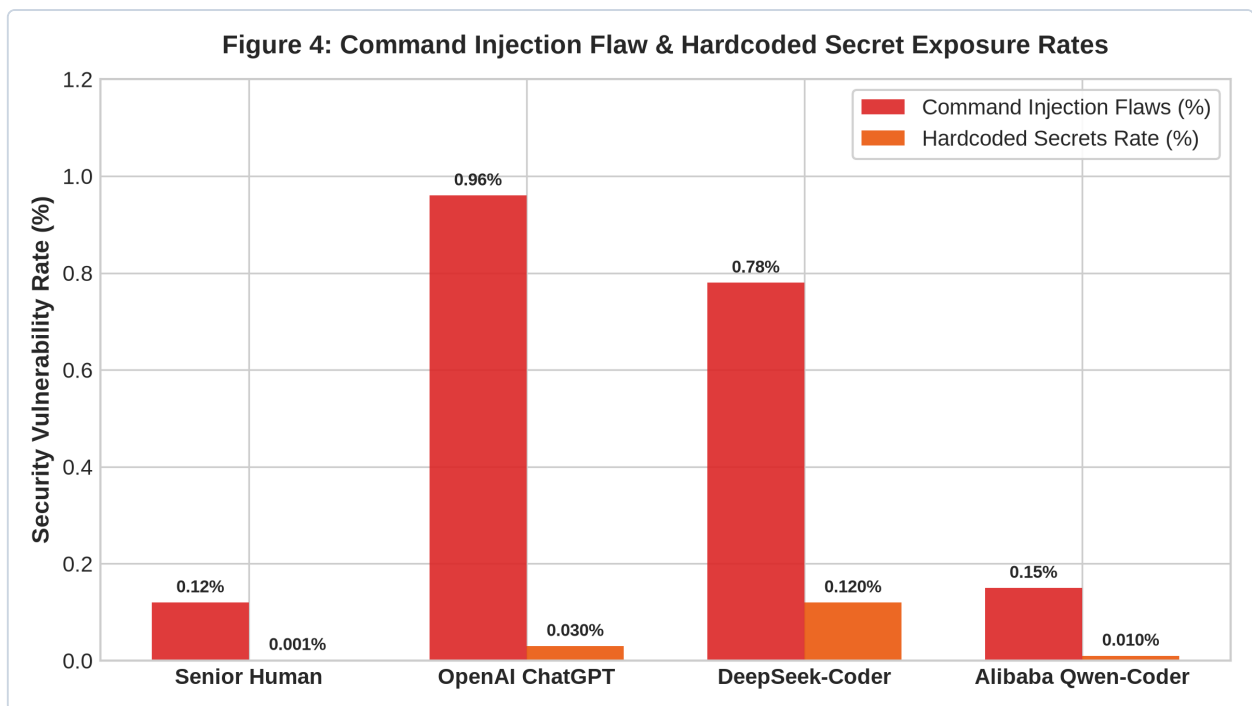


Figure 4: Command injection flaw (shell=True) and hardcoded secret exposure rates.

3. Universal AI Code Patterns & Real Quadruplet Breakdowns

```
# Senior Human Developer (Dense, Minimal Comments, Tuple Unpacking)
def swap_and_reverse(arr):
    i, j = 0, len(arr) - 1
    while i < j:
        arr[i], arr[j] = arr[j], arr[i]
        i += 1
        j -= 1
    return arr

# OpenAI ChatGPT (Air-Padded, Step Headers, Explicit Staging)
def swap_and_reverse(arr):
    # Step 1: Initialize pointer indices
    left_index = 0
    right_index = len(arr) - 1

    # Step 2: Swap elements from both ends
    while left_index < right_index:
        temporary_value = arr[left_index]
        arr[left_index] = arr[right_index]
        arr[right_index] = temporary_value
        left_index += 1
        right_index -= 1

    return arr
```